

TP Lockscreen : Système de Sécurité et d'Authentification

Ce document est un récapitulatif technique concis et complet de l'application **MonProjetNative**. Conçu pour être directement exploitable dans vos rapports de TP ou lors de vos soutenances, il détaille l'architecture modulaire et les algorithmes implémentés.

1. Architecture Modulaire

L'application a été entièrement refactorisée pour séparer la couche de présentation (UI) de la logique métier (Hooks) et des constantes de configuration.

```
MonProjetNative/  
├── src/  
│   ├── constants/  
│   │   └── security.ts          # Constantes de configuration globale  
│   ├── utils/  
│   │   └── securityUtils.ts    # Fonctions pures de calcul (TP)  
│   ├── hooks/  
│   │   ├── useAudio.ts        # Gestion mémoire et lecture des fichiers .mp3  
│   │   ├── useBiometrics.ts   # Logique de capteur natif FaceID/TouchID  
│   │   ├── useLockout.ts      # Sécurité anti brute-force et timer de blocage  
│   │   └── useShakeDetection.ts # Analyse RxJS en temps réel de l'accéléromètre  
│   ├── styles/  
│   │   └── globalStyles.ts     # Fichier unique regroupant tous les styles UI  
│   ├── components/  
│   │   ├── LockedView.tsx     # Composant de présentation : Écran verrouillé  
│   │   └── UnlockedView.tsx   # Composant de présentation : Écran d'accueil  
└── App.tsx                    # Chef d'orchestre (Point d'entrée de l'app)
```

2. Les 3 Piliers de l'Authentification

A. L'Authentification Biométrique (**useBiometrics.ts**)

- **Fonctionnement** : Utilise `react-native-biometrics` pour interroger le matériel de l'appareil via `isSensorAvailable()` (TouchID/FaceID) et invoquer le prompt natif d'authentification (`simplePrompt()`).
- **Sécurité anti-spam** : L'état `isBiometricPending` bloque les requêtes simultanées en cas de double tap.
- **Gestion d'erreur intelligente** : En cas d'erreur système (ex: pas d'empreinte enregistrée sur l'émulateur), l'erreur est interceptée et **aucune pénalité n'est appliquée** au compteur d'échecs. Seul un échec de validation de l'empreinte par l'utilisateur incrémente la pénalité.

B. La Détection de Secousses - "Shake" (useShakeDetection.ts)

- **Traitement de Signal (RxJS)** : Écoute en continu le capteur d'accéléromètre matériel (`accelerometer`) échantillonné toutes les 100ms et applique un filtre dynamique :
 1. **Calcul de la magnitude (norme euclidienne 3D)** : $\text{Force} = \sqrt{x^2 + y^2 + z^2}$ (en m/s^2). Cette valeur s'affranchit complètement de l'orientation du smartphone dans l'espace.
 2. **Filtrage des forces** : Seuls les mouvements dépassant le seuil `SHAKE_THRESHOLD` (22 m/s^2) sont conservés.
 3. **Anti-rebond (Debouncing)** : Un délai de `SHAKE_TIMEOUT` (800ms) vérifié par timestamp empêche de compter plusieurs secousses en un seul mouvement ample.
- **Formule dynamique du TP (securityUtils.ts)** : Le nombre de secousses exigé varie selon le jour de la semaine grâce à la formule :
 $\text{Secousses} = (\text{Jour de la semaine})^2 \bmod 5$
(Dimanche = 7. Le vendredi (jour 5) contourne le verrouillage pour un accès direct libre).

C. La Protection Brute-Force & Lockout (useLockout.ts)

- **Fonctionnement** : Si l'utilisateur accumule `MAX_ATTEMPTS` (4 échecs) successifs (biométrie ou abandon du mode secousse), l'application applique une pénalité de blocage temporaire de `LOCKOUT_DURATION` (60 secondes).
- **Pendant le blocage** : Les boutons d'authentification sont totalement désactivés, l'interface affiche un compte à rebours de secondes entières (`secondsRemaining`), et une alarme sonore continue (`alarm.mp3`) est déclenchée.
- **Résolution technique (Stale Closure)** : Un `useRef` (`lockoutUntilRef`) est utilisé pour synchroniser l'état de fin de blocage. Cela permet au timer asynchrone (`setInterval`) de lire la valeur temporelle exacte sans subir le problème de closure obsolète propre à React.

3. Robustesse Audio (useAudio.ts)

Pour éviter de ralentir le thread d'affichage (UI Thread) ou de faire planter le smartphone, la gestion audio est ultra-sécurisée :

- **Zéro fuite mémoire** : Les fichiers `.mp3` sont chargés au montage de l'application et impérativement détruits de la mémoire vive (`.release()`) au démontage de celle-ci.
- **Verrous d'exclusion mutuelle (Mutex)** : Des références `isErrorSoundPlaying` et `isAlarmSoundPlaying` garantissent qu'un son en cours de lecture ne se superpose pas avec lui-même en cas de déclenchements ultra-rapides successifs de l'utilisateur.

4. Pourquoi cette architecture est robuste ?

1. **Flux de Données Unidirectionnel** : Le point d'entrée `App.tsx` instancie tous les hooks et passe les données vers le bas aux composants graphiques sous forme de props en lecture seule.
2. **Séparation stricte des responsabilités (SoC)** : Si vous devez changer un style CSS, vous modifiez

`globalStyles.ts`. Si vous devez changer le seuil matériel du capteur, vous modifiez `security.ts`. Si vous modifiez l'affichage de l'écran, vous modifiez `LockedView.tsx`.

3. **Aucune dépendance cyclique** : Les hooks communiquent par callbacks asynchrones et réagissent proprement aux changements d'états descendants (ex: `useShakeDetection` s'auto-réinitialise en écoutant l'état `isLocked`).